دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش ۴

توصیف رفتاری

Digital Systems Design Laboratory

اعضای گروه

پوریا عرفانزاده ۴۰۱۱۱۰۹۱۸
فاطیما تیمارچی ۴۰۲۱۰۵۸۰۲
آیه صابری ۴۰۲۱۰۶۱۴۵

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

استاد درس

دکتر علیرضا اجلالی

سال تحصیلی ۱۴۰۴–۱۴۰۵

۱.۴ مقدمه و اهداف

هدف این آزمایش، آشنایی با توصیف رفتاری[۱] در زبان توصیف سخت‌افزار Verilog است. مطابق دستورکار، باید یک پشته[۲] با عمق ۸ و پهنای ۴ بیت طراحی شود که ورودی‌های ساعت، بازنشانی، دادهٔ ورودی و دو فرمان درج و برداشت، و خروجی‌های دادهٔ خروجی و دو نشانک پر و خالی داشته باشد.

پشته یک ساختار دادهٔ آخرین‌ورودی نخستین‌خروجی[۳] است؛ یعنی هر برداشت، همان مقداری را بازمی‌گرداند که در آخرین درج نوشته شده است. برخلاف آزمایش سوم که استفاده از هر توصیفی جز جریان داده ممنوع بود، در این آزمایش دقیقاً عکس آن خواسته شده است: مدار باید با بلوک رویه‌ای always، ساختار case و انتساب غیرمسدودکننده[۴] توصیف شود و رفتار مطلوب، نه ساختار دروازه‌ای آن، بیان گردد.

روند انجام آزمایش

ابتدا معنای دقیق هر پایه و رفتار مورد انتظار در حالت‌های مرزی مشخّص و تصمیم‌های طراحی مستند شد. سپس پیمانهٔ پشته به‌صورت رفتاری و پارامتری نوشته شد و برای آن یک میز آزمون[۵] خودآزما[۶] ساخته شد که هفت سناریوی کارکردی و مرزی را می‌آزماید. اجرای شبیه‌سازی در ModelSim با پرونده‌های فرمان خودکار انجام شد؛ یک بار با پیکربندی خواسته‌شدهٔ دستورکار و یک بار با پیکربندی دیگر برای نشان دادن پارامتری بودن واقعی طرح. در پایان، همان پروندهٔ سنتزپذیر در Quartus II همگردانی شد و نمای سطح انتقال ثبات‌ها[۷]، نماهای فناوری[۸]، گزارش مصرف منابع و تحلیل زمانی طرح بررسی شد.

۲.۴ مشخّصات خواسته‌شده و تصمیم‌های طراحی

الف) واسط پیمانه

جدول زیر پایه‌های خواسته‌شده در دستورکار و نقش هر یک را گردآوری می‌کند.

[1] Behavioral Modeling
[2] Stack
[3] Last In, First Out (LIFO)
[4] Non-Blocking Assignment
[5] Testbench
[6] Self-Checking Testbench
[7] Register Transfer Level (RTL) View
[8] Technology Map Viewer

نام	جهت	پهنا	نقش
Clk	ورودی	۱	ساعت مدار؛ همهٔ تغییرات در لبهٔ بالارونده
RstN	ورودی	۱	بازنشانی ناهم‌زمان و فعّال‌صفر
Data_In	ورودی	۴	داده‌ای که باید در پشته درج شود
Push	ورودی	۱	فرمان درج
Pop	ورودی	۱	فرمان برداشت
Data_Out	خروجی	۴	داده‌ای که از پشته برداشته شده است
Full	خروجی	۱	مقدار یک یعنی پشته پر است
Empty	خروجی	۱	مقدار **صفر** یعنی پشته خالی است

جدول ۱: پایه‌های پشته مطابق دستورکار

ب) قرارداد نشانک خالی

یک نکتهٔ مهم که باید پیش از خواندن کد و شکل موج‌ها روشن شود، قطبیّت نشانک Empty است. دستورکار به‌صراحت نوشته است Empty=0 indicates that the stack is empty؛ یعنی این نشانک برخلاف انتظار متعارف، فعّال‌صفر است و هنگام خالی بودن پشته مقدار صفر و هنگام وجود دست‌کم یک عنصر مقدار یک می‌گیرد. در مقابل، نشانک Full قطبیّت متعارف دارد و هنگام پر بودن پشته یک می‌شود.

این قرارداد در سراسر طرح و میز آزمون رعایت شده است. بنابراین در شکل موج‌ها، بالا بودن Empty به معنای «پشته خالی نیست» خوانده می‌شود و شرط if (Empty) در کد، دقیقاً همان شرط «خالی نبودن پشته» است. برای پرهیز از ابهام، در متن این گزارش هرجا لازم بوده، معنای منطقی هر مقدار نیز آورده شده است.

ج) حالت‌های تعریف‌نشده در دستورکار

دستورکار رفتار مدار را در چند وضعیّت مرزی مشخّص نکرده است. تصمیم‌های زیر گرفته و در کد و میز آزمون پیاده و آزموده شدند:

- **درج در پشتهٔ پر:** فرمان نادیده گرفته می‌شود. اشاره‌گر پشته[9] و محتوای حافظه دست‌نخورده می‌مانند تا داده‌های معتبر بازنویسی نشوند.
- **برداشت از پشتهٔ خالی:** فرمان نادیده گرفته می‌شود و Data_Out مقدار پیشین خود را نگه می‌دارد؛

[9] Stack Pointer

زیرا هیچ دادهٔ معتبری برای بازگرداندن وجود ندارد و صفر کردن خروجی می‌توانست با یک دادهٔ واقعی صفر اشتباه گرفته شود.

- **درج و برداشت هم‌زمان:** به‌عنوان یک تصمیم طراحی، هیچ عملی انجام نمی‌شود. این انتخاب ساده‌ترین و قابل‌آزمون‌ترین رفتار ممکن است و از تعریف اولویّت دلبخواهی میان دو فرمان پرهیز می‌کند.
- **بازنشانی:** ناهم‌زمان و فعّال‌صفر است. تنها اشاره‌گر پشته و Data_Out صفر می‌شوند و آرایهٔ حافظه عمداً بازنشانی نمی‌شود؛ زیرا اشاره‌گر تضمین می‌کند هیچ‌گاه خانه‌ای خوانده نشود که پیش‌تر نوشته نشده باشد و بازنشانی ۳۲ فلیپ‌فلاپ حافظه تنها هزینهٔ اضافی می‌داشت.

چهار ترکیب ممکن دو فرمان و رفتار مدار در برابر هر یک، در جدول زیر خلاصه شده است.

رفتار مدار	Pop	Push
بدون فرمان؛ وضعیّت حفظ می‌شود	0	0
درج در بالای پشته، تنها اگر پشته پر نباشد	0	1
برداشت از بالای پشته، تنها اگر پشته خالی نباشد	1	0
بی‌اثر؛ هیچ عملی انجام نمی‌شود	1	1

جدول ۲: جدول تصمیم فرمان‌های درج و برداشت

۳.۴ توصیف رفتاری پشته

الف) اشاره‌گر پشته و پهنای آن

هستهٔ طرح، ثبات Stack_Pointer است که تعداد عناصر معتبر پشته را نگه می‌دارد. با این تعریف، همین مقدار هم‌زمان نشانی نخستین خانهٔ آزاد نیز هست؛ ازاین‌رو درج در خانه‌ای از آرایهٔ Stack_Mem انجام می‌شود که نشانی آن برابر مقدار جاری اشاره‌گر است و سپس اشاره‌گر یک واحد افزایش می‌یابد؛ و برداشت از خانه‌ای انجام می‌شود که نشانی آن یک واحد کمتر از اشاره‌گر است و سپس اشاره‌گر یک واحد کاسته می‌شود. این انتخاب، منطق دو عمل را متقارن و ساده نگه می‌دارد.

پهنای اشاره‌گر با رابطهٔ

$$\mathrm{PTR_WIDTH} = \left\lceil \log_2(\mathrm{DEPTH} + 1) \right\rceil$$

محاسبه می‌شود و برای عمق ۸ برابر چهار بیت است. افزودن یک واحد به عمق در این رابطه ضروری است، زیرا اشاره‌گر باید بتواند $\mathrm{DEPTH} + 1$ مقدار متمایز، یعنی صفر تا ۸، را نمایش دهد. اگر به‌اشتباه

از $\lceil \log_2 8 \rceil = 3$ بیت استفاده می‌شد، مقدار ۸ که همان شرط پر بودن پشته است اصلاً نمایش‌پذیر نبود و شمارنده به‌جای رسیدن به آن، به صفر بازمی‌گشت.

دو نشانک وضعیّت تنها از روی اشاره‌گر و به‌صورت ترکیبی ساخته می‌شوند:

$$\text{Full} = \big(\text{Stack_Pointer} = \text{DEPTH}\big), \qquad \text{Empty} = \big(\text{Stack_Pointer} \neq 0\big).$$

وابسته بودن نشانک‌ها به یک منبع یگانه، امکان ناسازگاری میان وضعیّت واقعی پشته و نشانک‌های گزارش‌شده را از میان می‌برد.

ب) بلوک رویه‌ای

کلّ رفتار مدار در یک بلوک always حسّاس به لبهٔ بالارونده‌ٔ ساعت و لبهٔ پایین‌رونده‌ٔ بازنشانی توصیف شده است. حضور negedge RstN در فهرست حسّاسیّت همان چیزی است که بازنشانی را ناهم‌زمان[10] می‌کند. درون بلوک، ساختار case روی الحاق دو بیت فرمان، چهار حالت جدول تصمیم را از هم جدا می‌کند و همهٔ انتساب‌ها از نوع غیرمسدودکننده‌اند تا رفتار ثبات‌های هم‌زمان به‌درستی مدل شود.

طرح با دو پارامتر[11] DEPTH و WIDTH نوشته شده است. مقادیر پیش‌فرض آن‌ها همان ۸ و ۴ خواسته‌شدهٔ دستورکار است، امّا هیچ جای کد به این دو عدد وابستگی سخت ندارد؛ این ادّعا در بخش شبیه‌سازی با اجرای پیکربندی دوم راستی‌آزمایی می‌شود.

```
`timescale 1ns/1ps

module stack #(
    parameter DEPTH = 8,
    parameter WIDTH = 4
)(
    input                  Clk,
    input                  RstN,
    input      [WIDTH-1:0] Data_In,
    input                  Push,
    input                  Pop,
    output reg [WIDTH-1:0] Data_Out,
    output                 Full,
    output                 Empty
);

    // Stack_Pointer represents the number of valid elements and therefore
    // also points to the next free location. It must represent 0..DEPTH.
    localparam PTR_WIDTH = (DEPTH < 2) ? 1 : $clog2(DEPTH + 1);

    reg [WIDTH-1:0] Stack_Mem [0:DEPTH-1];
    reg [PTR_WIDTH-1:0] Stack_Pointer;

    // Status flags are derived only from the pointer.
    assign Full  = (Stack_Pointer == DEPTH);
    assign Empty = (Stack_Pointer == 0) ? 1'b0 : 1'b1;

```

[10] Asynchronous Reset

[11] Parameter

```
    always @(posedge Clk or negedge RstN) begin
        if (!RstN) begin
            Stack_Pointer <= {PTR_WIDTH{1'b0}};
            Data_Out      <= {WIDTH{1'b0}};
        end
        else begin
            case ({Push, Pop})
                // Push only
                2'b10: begin
                    if (!Full) begin
                        Stack_Mem[Stack_Pointer] <= Data_In;
                        Stack_Pointer            <= Stack_Pointer + 1'b1;
                    end
                end

                // Pop only
                2'b01: begin
                    if (Empty) begin
                        Data_Out      <= Stack_Mem[Stack_Pointer - 1'b1];
                        Stack_Pointer <= Stack_Pointer - 1'b1;
                    end
                end

                // 00: no command
                // 11: simultaneous Push and Pop -> no operation
                default: begin
                    // Hold current state.
                end
            endcase
        end
    end

endmodule
```

کد ۱: توصیف رفتاری پشته

۴.۴ میز آزمون خودآزما

میز آزمون دو وظیفه[۱۲] کمکی دارد. وظیفهٔ push_value داده را در لبهٔ پایین‌رونده اعمال و فرمان درج را برای دقیقاً یک لبهٔ فعّال فعّال می‌کند؛ وظیفهٔ pop_and_check یک برداشت انجام می‌دهد و مقدار بازگشتی را با مقدار انتظاری مقایسه و هر مغایرت را در شمارندهٔ خطا ثبت می‌کند. اعمال ورودی‌ها در لبهٔ پایین‌رونده و نمونه‌برداری با تأخیر یک نانوثانیه پس از لبهٔ بالارونده، از نقض شرط‌های برپایی و نگهداری و از رقابت خواندن و نوشتن در لحظهٔ لبه جلوگیری می‌کند.
آزمون از هفت مرحلهٔ پیاپی تشکیل شده است:

- **مرحلهٔ ۱ — بازنشانی**: پس از دو پالس ساعت با RstN فعّال، بررسی می‌شود که پشته خالی باشد، نشانک پر فعّال نباشد و Data_Out صفر باشد.
- **مرحلهٔ ۲ — برداشت از پشتهٔ خالی**: یک فرمان برداشت اعمال و بررسی می‌شود که وضعیّت پشته تغییر نکند.

[12] Task

- **مرحلهٔ ۳ — پر کردن کامل:** دقیقاً DEPTH مقدار پیاپی درج و پس از آن، فعّال شدن نشانک پر بررسی می‌شود.
- **مرحلهٔ ۴ — درج در پشتهٔ پر:** یک درج دیگر اعمال و بررسی می‌شود که نشانک پر همچنان فعّال بماند و پشته سرریز نکند.
- **مرحلهٔ ۵ — برداشت کامل و بررسی ترتیب:** همهٔ مقادیر برداشته و ترتیب آخرین‌ورودی نخستین‌خروجی آن‌ها یک‌به‌یک با مقدار انتظاری سنجیده می‌شود.
- **مرحلهٔ ۶ — برداشت دوبارهٔ از پشتهٔ خالی:** افزون بر وضعیّت پشته، بررسی می‌شود که Data_Out نیز مقدار معتبر پیشین خود را نگه دارد.
- **مرحلهٔ ۷ — درج و برداشت هم‌زمان:** پس از یک درج، هر دو فرمان با هم فعّال می‌شوند. در این مرحله، افزون بر خروجی‌ها، مقدار اشاره‌گر درونی نیز مستقیماً وارسی می‌شود.

وارسی مستقیم اشاره‌گر در مرحلهٔ هفتم تصمیم آگاهانه‌ای است. اگر تنها به خروجی‌ها بسنده می‌شد، مداری که به‌اشتباه هم درج و هم برداشت را انجام می‌دهد نیز از آزمون سربلند بیرون می‌آمد، چون اثر دو عمل بر اشاره‌گر یکدیگر را خنثی می‌کند. سنجش مستقیم Stack_Pointer این راه فرار را می‌بندد.

```
`timescale 1ns/1ps


module tb_stack #(
    parameter DEPTH = 8,
    parameter WIDTH = 4
);

    reg                 Clk;
    reg                 RstN;
    reg  [WIDTH-1:0]    Data_In;
    reg                 Push;
    reg                 Pop;

    wire [WIDTH-1:0]    Data_Out;
    wire                Full;
    wire                Empty;

    integer errors;
    integer i;
    reg [WIDTH-1:0] saved_data_out;

    stack #(
        .DEPTH(DEPTH),
        .WIDTH(WIDTH)
    ) dut (
        .Clk     (Clk),
        .RstN    (RstN),
        .Data_In (Data_In),
        .Push    (Push),
        .Pop     (Pop),
        .Data_Out(Data_Out),
        .Full    (Full),
        .Empty   (Empty)
    );

```

```
    // 10 ns clock period
    always #5 Clk = ~Clk;

    task push_value;
        input [WIDTH-1:0] value;
        begin
            @(negedge Clk);
            Data_In = value;
            Push    = 1'b1;
            Pop     = 1'b0;

            @(posedge Clk);
            #1;
            Push = 1'b0;

            $display("PUSH %0d | Full=%b Empty=%b", value, Full, Empty);
        end
    endtask

    task pop_and_check;
        input [WIDTH-1:0] expected;
        begin
            @(negedge Clk);
            Push = 1'b0;
            Pop  = 1'b1;

            @(posedge Clk);
            #1;
            Pop = 1'b0;

            if (Data_Out !== expected) begin
                errors = errors + 1;
                $display("ERROR: POP expected=%0d, got=%0d", expected, Data_Out);
            end
            else begin
                $display("POP  %0d | Full=%b Empty=%b", Data_Out, Full, Empty);
            end
        end
    endtask

    initial begin
        Clk     = 1'b0;
        RstN    = 1'b0;
        Data_In = {WIDTH{1'b0}};
        Push    = 1'b0;
        Pop     = 1'b0;
        errors  = 0;

        $display("========================================");
        $display("Stack test started: DEPTH=%0d WIDTH=%0d", DEPTH, WIDTH);
        $display("========================================");

        // 1) Reset
        repeat (2) @(posedge Clk);
        #1;
        if ((Full !== 1'b0) || (Empty !== 1'b0) ||
            (Data_Out !== {WIDTH{1'b0}})) begin
            errors = errors + 1;
            $display("ERROR: Reset state is incorrect.");
        end

        @(negedge Clk);
        RstN = 1'b1;

        // 2) Pop while empty -> ignored
        Pop = 1'b1;
        @(posedge Clk);
        #1;
        Pop = 1'b0;
        if ((Empty !== 1'b0) || (Full !== 1'b0)) begin
            errors = errors + 1;
            $display("ERROR: Empty-pop changed stack status.");
```

```
        end

        // 3) Fill exactly DEPTH locations
        for (i = 1; i <= DEPTH; i = i + 1)
            push_value(i);

        if ((Full !== 1'b1) || (Empty !== 1'b1)) begin
            errors = errors + 1;
            $display("ERROR: Full flag is incorrect after %0d pushes.", DEPTH);
        end

        // 4) Push while full -> ignored
        push_value({WIDTH{1'b1}});
        if (Full !== 1'b1) begin
            errors = errors + 1;
            $display("ERROR: Push while full changed stack status.");
        end

        // 5) Pop everything and verify LIFO order
        for (i = DEPTH; i >= 1; i = i - 1)
            pop_and_check(i);

        if ((Full !== 1'b0) || (Empty !== 1'b0)) begin
            errors = errors + 1;
            $display("ERROR: Stack is not empty after all pops.");
        end

        // 6) Pop while empty again -> ignored.
        //    Data_Out must also keep its previous valid value.
        saved_data_out = Data_Out;

        @(negedge Clk);
        Pop = 1'b1;
        @(posedge Clk);
        #1;
        Pop = 1'b0;

        if ((Empty !== 1'b0) || (Full !== 1'b0) ||
            (Data_Out !== saved_data_out)) begin
            errors = errors + 1;
            $display("ERROR: Empty-pop changed the stack or Data_Out.");
        end

        // 7) Simultaneous Push & Pop -> no operation by design.
        //    Use the internal pointer as an additional invariant check so
        //    this test cannot falsely pass if the DUT accidentally performs
        //    only a push or only a pop.
        push_value({WIDTH{1'b1}});

        @(negedge Clk);
        Data_In = {WIDTH{1'b0}};
        Push    = 1'b1;
        Pop     = 1'b1;
        @(posedge Clk);
        #1;
        Push = 1'b0;
        Pop  = 1'b0;

        if (dut.Stack_Pointer !== 1) begin
            errors = errors + 1;
            $display("ERROR: Simultaneous Push/Pop changed Stack_Pointer.");
        end

        // The previous top value must still be present.
        pop_and_check({WIDTH{1'b1}});

        $display("========================================");
        if (errors == 0)
            $display("ALL TESTS PASSED");
        else
            $display("TEST FAILED: %0d error(s)", errors);
        $display("========================================");
```

```
181
182         $finish;
183     end
184
185 endmodule
```

کد ۲: میز آزمون خودآزمای پشته

۵.۴ اجرا در ModelSim

برای آنکه اجرای شبیه‌سازی تکرارپذیر باشد و به دنبالهٔ فرمان‌های دستی وابسته نماند، سه پروندهٔ فرمان نوشته شد. پروندهٔ run.do کتابخانهٔ پیشین را پاک و از نو می‌سازد، دو پروندهٔ توصیف سخت‌افزار را همگردانی می‌کند، شبیه‌ساز را روی میز آزمون بارگذاری و سپس پنجرهٔ موج را می‌سازد و شبیه‌سازی را تا پایان اجرا می‌کند. گزینهٔ -voptargs=+acc از بهینه‌سازی سیگنال‌های داخلی جلوگیری می‌کند؛ بدون آن، اشاره‌گر و آرایهٔ حافظه در پنجرهٔ موج در دسترس نمی‌بودند و مرحلهٔ هفتم آزمون نیز نمی‌توانست اشاره‌گر را وارسی کند.

```
transcript on

if {[file exists work]} {
    vdel -lib work -all
}

vlib work
vmap work work

vlog stack.v
vlog tb_stack.v

vsim -voptargs=+acc work.tb_stack

do wave.do
run -all
```

کد ۳: پرونده‌ی فرمان اجرای شبیه‌سازی با پیکربندی پیش‌فرض

پروندهٔ wave.do سیگنال‌ها را در سه دستهٔ ورودی، خروجی و درونی به پنجرهٔ موج می‌افزاید و نمایش مقادیر عددی را به مبنای ده تغییر می‌دهد تا دنبال‌کردن مقادیر پشته آسان‌تر شود.

```
quietly WaveActivateNextPane {} 0
add wave -divider {INPUTS}
add wave sim:/tb_stack/Clk
add wave sim:/tb_stack/RstN
add wave sim:/tb_stack/Push
add wave sim:/tb_stack/Pop
add wave -radix unsigned sim:/tb_stack/Data_In

add wave -divider {OUTPUTS}
add wave -radix unsigned sim:/tb_stack/Data_Out
add wave sim:/tb_stack/Full
add wave sim:/tb_stack/Empty
```

```
add wave -divider {INTERNAL}
add wave -radix unsigned sim:/tb_stack/dut/Stack_Pointer
add wave sim:/tb_stack/dut/Stack_Mem

configure wave -namecolwidth 180
configure wave -valuecolwidth 90
configure wave -justifyvalue left
TreeUpdate [SetDefaultTree]
WaveRestoreZoom {0 ns} {300 ns}
```

کد ۴: پرونده‌ی پیکربندی پنجره‌ی موج

پروندهٔ سوم، run_param.do، همان طرح و همان میز آزمون را با پیکربندی دیگری اجرا می‌کند: عمق ۴ و پهنای ۸ بیت. مقادیر پارامترها با گزینهٔ g- در زمان بارگذاری شبیه‌ساز جایگزین می‌شوند و هیچ تغییری در کد لازم نیست.

```
transcript on

if {[file exists work]} {
    vdel -lib work -all
}

vlib work
vmap work work

vlog stack.v
vlog tb_stack.v

# Second configuration to demonstrate that the design is truly parameterized.
# DEPTH=4, WIDTH=8
vsim -voptargs=+acc -gDEPTH=4 -gWIDTH=8 work.tb_stack

do wave.do
run -all
```

کد ۵: پرونده‌ی فرمان اجرای پیکربندی دوم برای راستی‌آزمایی پارامتری بودن طرح

تصویر زیر آغاز اجرای run.do را نشان می‌دهد: ساخت کتابخانه، همگردانی موفّق دو پیمانه، بارگذاری شبیه‌ساز و افزودن سیگنال‌ها به پنجرهٔ موج.



شکل ۱: آغاز اجرای پرونده‌ی فرمان و همگردانی موفّق پیمانه‌ها

خروجی کارکردی آزمون در ادامهٔ همان رونوشت چاپ می‌شود. هر سطر، فرمان انجام‌شده و مقدار دو نشانک را پس از آن گزارش می‌کند و در پایان پیام ALL TESTS PASSED نشان می‌دهد که شمارندهٔ خطا صفر مانده است.

```
# add wave sim:/tb_stack/Empty
#
# add wave -divider {INTERNAL}
# add wave -radix unsigned sim:/tb_stack/dut/Stack_Pointer
# add wave sim:/tb_stack/dut/Stack_Mem
#
# configure wave -namecolwidth 180
# configure wave -valuecolwidth 90
# configure wave -justifyvalue left
# TreeUpdate [SetDefaultTree]
# WaveRestoreZoom {0 ns} {300 ns}
# run -all
# ========================================
# Stack test started: DEPTH=8 WIDTH=4
# ========================================
# PUSH 1 | Full=0 Empty=1
# PUSH 2 | Full=0 Empty=1
# PUSH 3 | Full=0 Empty=1
# PUSH 4 | Full=0 Empty=1
# PUSH 5 | Full=0 Empty=1
# PUSH 6 | Full=0 Empty=1
# PUSH 7 | Full=0 Empty=1
# PUSH 8 | Full=1 Empty=1
# PUSH 15 | Full=1 Empty=1
# POP  8 | Full=0 Empty=1
# POP  7 | Full=0 Empty=1
# POP  6 | Full=0 Empty=1
# POP  5 | Full=0 Empty=1
# POP  4 | Full=0 Empty=1
# POP  3 | Full=0 Empty=1
# POP  2 | Full=0 Empty=1
# POP  1 | Full=0 Empty=0
# PUSH 15 | Full=0 Empty=1
# POP  15 | Full=0 Empty=0
# ========================================
# ALL TESTS PASSED
# ========================================
# ** Note: $finish    : tb_stack.v(185)
#    Time: 236 ns  Iteration: 0  Instance: /tb_stack
# 1
# Break in Module tb_stack at tb_stack.v line 185
# Simulation Breakpoint: 1
# Break in Module tb_stack at tb_stack.v line 185
# MACRO ./run.do PAUSED at line 16
```

شکل ۲: خروجی کامل آزمون با پیکربندی عمق ۸ و پهنای ۴ بیت

دنبالهٔ چاپ‌شده دقیقاً همان است که انتظار می‌رفت: هشت درج پیاپی با مقادیر ۱ تا ۸، سپس یک درج نادیده‌گرفته‌شده در پشتهٔ پر، سپس هشت برداشت با ترتیب معکوس ۸ تا ۱، و سرانجام یک درج و یک برداشت پایانی برای آزمون فرمان هم‌زمان. سطر PUSH 8 نخستین سطری است که در آن $\text{Full} = 1$ گزارش شده و سطر پایانی POP 1 نخستین سطری است که در آن نشانک خالی دوباره صفر، یعنی پشته خالی، شده است.

۶.۴ تحلیل شکل موج‌ها

الف) سیگنال‌های افزوده‌شده به پنجره‌ی موج

جدول زیر ده سیگنال نمایش‌داده‌شده در پنجرهٔ موج و معنای آن‌ها را گردآوری می‌کند.

دسته	سیگنال	معنا
ورودی	Clk	ساعت با دورهٔ ۱۰ نانوثانیه
ورودی	RstN	بازنشانی ناهم‌زمان فعّال‌صفر
ورودی	Push	فرمان درج
ورودی	Pop	فرمان برداشت
ورودی	Data_In	دادهٔ ورودی، در مبنای ده
خروجی	Data_Out	دادهٔ برداشته‌شده، در مبنای ده
خروجی	Full	یک یعنی پشته پر است
خروجی	Empty	صفر یعنی پشته خالی است
درونی	Stack_Pointer	شمار عناصر معتبر و نشانی خانهٔ آزاد بعدی
درونی	Stack_Mem	آرایهٔ حافظهٔ پشته

جدول ۳: سیگنال‌های پنجره‌ی موج و معنای آن‌ها

ب) دنباله‌ی کامل رویدادها در پیکربندی خواسته‌شده

چون ساعت میز آزمون دورهٔ ۱۰ نانوثانیه دارد و نخستین لبهٔ بالارونده در لحظهٔ ۵ نانوثانیه رخ می‌دهد، لبه‌های فعّال در لحظه‌های $5, 15, 25, \ldots$ قرار می‌گیرند و می‌توان هر رویداد شکل موج را به یک لبهٔ مشخّص نسبت داد. جدول زیر کلّ شبیه‌سازی را لبه‌به‌لبه دنبال می‌کند؛ مقادیر ستون‌ها، وضعیّت پس از هر لبه‌اند.

Empty	Full	Data_Out	اشاره‌گر	فرمان	زمان
0	0	0	0	بازنشانی فعّال	5, 15
0	0	0	0	برداشت از خالی	25
1	0	0	1	درج 1	35
1	0	0	2	درج 2	45
1	0	0	3	درج 3	55
1	0	0	4	درج 4	65
1	0	0	5	درج 5	75
1	0	0	6	درج 6	85
1	0	0	7	درج 7	95
1	1	0	8	درج 8	105
1	1	0	8	درج 15 در پشتهٔ پر	115
1	0	8	7	برداشت	125
1	0	7	6	برداشت	135
1	0	6	5	برداشت	145
1	0	5	4	برداشت	155
1	0	4	3	برداشت	165
1	0	3	2	برداشت	175
1	0	2	1	برداشت	185
0	0	1	0	برداشت	195
0	0	1	0	برداشت از خالی	205
1	0	1	1	درج 15	215
1	0	1	1	درج و برداشت هم‌زمان	225
0	0	15	0	برداشت	235

جدول ۴: دنباله‌ی کامل رویدادهای شبیه‌سازی با عمق ۸ و پهنای ۴ بیت؛ زمان‌ها بر حسب نانوثانیه



شکل ۳: شکل موج کامل شبیه‌سازی با عمق ۸ و پهنای ۴ بیت

ج) تفسیر تک‌تک ویژگی‌های شکل موج

با کنار هم گذاشتن جدول بالا و شکل موج، همهٔ ویژگی‌های دیدنی در نما توضیح‌پذیر می‌شوند:

- **بازهٔ بازنشانی**: تا لحظهٔ ۲۰ نانوثانیه RstN پایین است و اشاره‌گر و Data_Out روی صفر می‌مانند. چون بازنشانی ناهم‌زمان است، اشاره‌گر حتّی پیش از نخستین لبهٔ فعّال نیز مقدار معیّن صفر دارد و هیچ بازهٔ نامعیّنی در آن دیده نمی‌شود.

- **مقدار نامعیّن آرایهٔ حافظه**: ردیف Stack_Mem در آغاز شبیه‌سازی مقدار xxxx دارد و تنها خانه‌هایی که نوشته می‌شوند به‌تدریج معیّن می‌گردند. این دقیقاً همان تصمیم طراحی است که در بخش ۲.۴ توضیح داده شد: آرایهٔ حافظه بازنشانی نمی‌شود، امّا چون اشاره‌گر هرگز به خانهٔ نانوشته اجازهٔ خواندن نمی‌دهد، این مقدار نامعیّن هیچ‌گاه به خروجی راه نمی‌یابد.

- **پلّهٔ صعودی اشاره‌گر**: میان لحظه‌های ۳۵ تا ۱۰۵، اشاره‌گر در هشت پلّهٔ یکسان از صفر به ۸ می‌رسد و ردیف Stack_Mem به‌ترتیب مقادیر 0001 تا 1000 را در خود جای می‌دهد.

- **نشانک خالی**: Empty دقیقاً هم‌زمان با نخستین درج بالا می‌رود و تا لحظهٔ ۱۹۵ نانوثانیه که اشاره‌گر دوباره صفر می‌شود بالا می‌ماند. با توجّه به قرارداد فعّال‌صفر دستورکار، این یعنی «پشته از نخستین درج تا آخرین برداشت خالی نبوده است».

- **نشانک پر**: Full تنها در بازهٔ ۱۰۵ تا ۱۲۵ نانوثانیه بالاست؛ یعنی از لحظه‌ای که هشتمین عنصر درج می‌شود تا لحظه‌ای که نخستین برداشت انجام می‌گیرد.

- **درج نادیده‌گرفته‌شده**: در لحظهٔ ۱۱۵ نانوثانیه Push بالاست و Data_In برابر ۱۵ است، امّا اشاره‌گر روی ۸ ثابت می‌ماند و Stack_Mem تغییری نمی‌کند. این بخش، رفتار درست مدار در برابر سرریز[۱۳] را نشان می‌دهد.

- **پلّهٔ نزولی و ترتیب معکوس**: میان لحظه‌های ۱۲۵ تا ۱۹۵، اشاره‌گر در هشت پلّه به صفر بازمی‌گردد و Data_Out به‌ترتیب مقادیر ۸، ۷، ۶، ۵، ۴، ۳، ۲ و ۱ را نشان می‌دهد. این معکوس بودن ترتیب، همان ویژگی آخرین‌ورودی نخستین‌خروجی پشته است.

- **برداشت نادیده‌گرفته‌شده**: در لحظهٔ ۲۰۵ نانوثانیه Pop بالاست، امّا اشاره‌گر روی صفر می‌ماند و Data_Out همچنان مقدار ۱ را نگه می‌دارد؛ یعنی مدار در برابر تهی‌روی[۱۴] نیز پایدار است و خروجی خود را با داده‌ای نامعتبر آلوده نمی‌کند.

[13] Overflow
[14] Underflow

- **فرمان هم‌زمان**: در لحظهٔ ۲۲۵ نانوثانیه هر دو سیگنال Push و Pop با هم بالا هستند. اشاره‌گر روی ۱ ثابت می‌ماند و Data_Out تغییر نمی‌کند؛ سپس برداشت پایانی در لحظهٔ ۲۳۵ مقدار ۱۵ را بازمی‌گرداند، که ثابت می‌کند مقدار درج‌شده پیش از فرمان هم‌زمان دست‌نخورده باقی مانده بود.
- **تأخیر یک‌پالسی نشانک‌ها**: نشانک‌ها ترکیبی‌اند و از اشاره‌گر ساخته می‌شوند؛ ازاین‌رو وضعیّت پس از آخرین عمل انجام‌شده را نشان می‌دهند، نه وضعیّت میان لبه را. برای نمونه، Full در همان لبه‌ای بالا می‌رود که هشتمین درج در آن کامل می‌شود.

د) راستی‌آزمایی پارامتری بودن طرح

برای نشان دادن اینکه طرح به مقادیر ۸ و ۴ وابستگی سخت ندارد، همان کد و همان میز آزمون با عمق ۴ و پهنای ۸ بیت اجرا شد. رونوشت این اجرا نشان می‌دهد که مقادیر پارامترها در زمان بارگذاری جایگزین شده‌اند.

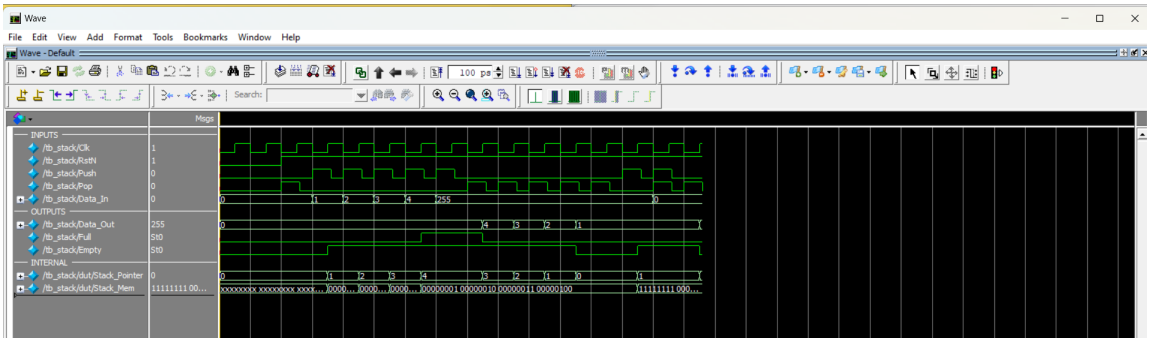
```
VSIM(paused)> do  run_param.do
# transcript on
#
# if {[file exists work]} {
#     vdel -lib work -all
# }
#
# vlib work
# vmap work work
# Modifying modelsim.ini
#
# vlog stack.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module stack
#
# Top level modules:
#       stack
# vlog tb_stack.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module tb_stack
#
# Top level modules:
#       tb_stack
#
# Second configuration to demonstrate that the design is truly parameterized.
# DEPTH=4, WIDTH=8
# vsim -voptargs=+acc -gDEPTH=4 -gWIDTH=8 work.tb_stack
# vsim -voptargs=+acc -gWIDTH=8 -gDEPTH=4 work.tb_stack
# Loading work.tb_stack
# Loading work.stack
#
# do wave.do
# quietly WaveActivateNextPane {} 0
# add wave -divider {INPUTS}
# add wave sim:/tb_stack/Clk
# add wave sim:/tb_stack/RstN
# add wave sim:/tb_stack/Push
# add wave sim:/tb_stack/Pop
# add wave -radix unsigned sim:/tb_stack/Data_In
#
# add wave -divider {OUTPUTS}
# add wave -radix unsigned sim:/tb_stack/Data_Out
# add wave sim:/tb_stack/Full
# add wave sim:/tb_stack/Empty
#
# add wave -divider {INTERNAL}
# add wave -radix unsigned sim:/tb_stack/dut/Stack_Pointer
```

شکل ۴: بارگذاری شبیه‌ساز با پارامترهای عمق ۴ و پهنای ۸ بیت

```
Transcript
File  Edit  View  Bookmarks  Window  Help
Transcript
# add wave sim:/tb_stack/RstN
# add wave sim:/tb_stack/Push
# add wave sim:/tb_stack/Pop
# add wave -radix unsigned sim:/tb_stack/Data_In
#
# add wave -divider {OUTPUTS}
# add wave -radix unsigned sim:/tb_stack/Data_Out
# add wave sim:/tb_stack/Full
# add wave sim:/tb_stack/Empty
#
# add wave -divider {INTERNAL}
# add wave -radix unsigned sim:/tb_stack/dut/Stack_Pointer
# add wave sim:/tb_stack/dut/Stack_Mem
#
# configure wave -namecolwidth 180
# configure wave -valuecolwidth 90
# configure wave -justifyvalue left
# TreeUpdate [SetDefaultTree]
# WaveRestoreZoom {0 ns} {300 ns}
# run -all
# ========================================
# Stack test started: DEPTH=4 WIDTH=8
# ========================================
# PUSH 1 | Full=0 Empty=1
# PUSH 2 | Full=0 Empty=1
# PUSH 3 | Full=0 Empty=1
# PUSH 4 | Full=1 Empty=1
# PUSH 255 | Full=1 Empty=1
# POP  4 | Full=0 Empty=1
# POP  3 | Full=0 Empty=1
# POP  2 | Full=0 Empty=1
# POP  1 | Full=0 Empty=0
# PUSH 255 | Full=0 Empty=1
# POP  255 | Full=0 Empty=0
# ========================================
# ALL TESTS PASSED
# ========================================
# ** Note: $finish    : tb_stack.v(185)
#    Time: 156 ns  Iteration: 0  Instance: /tb_stack
# 1
# Break in Module tb_stack at tb_stack.v line 185
# Simulation Breakpoint: 1
# Break in Module tb_stack at tb_stack.v line 185
# MACRO ./run_param.do PAUSED at line 18
```

شکل ۵: خروجی کامل آزمون با پیکربندی عمق ۴ و پهنای ۸ بیت

در این اجرا، پشته پس از چهار درج پر می‌شود، مقدار مرزی 255 که بزرگ‌ترین عدد نمایش‌پذیر در هشت بیت است به‌جای 15 به کار می‌رود، و کلّ آزمون در ۱۵۶ نانوثانیه به‌جای ۲۳۶ نانوثانیه پایان می‌یابد. هر هفت مرحلهٔ آزمون بدون تغییر در کد میز آزمون و بدون هیچ خطا اجرا شدند؛ این یعنی پارامتری بودن طرح صرفاً ادّعا نیست و در عمل نیز کار می‌کند. شکل موج این اجرا نیز همان الگوی پیشین را با مقیاس کوچک‌تر نشان می‌دهد.

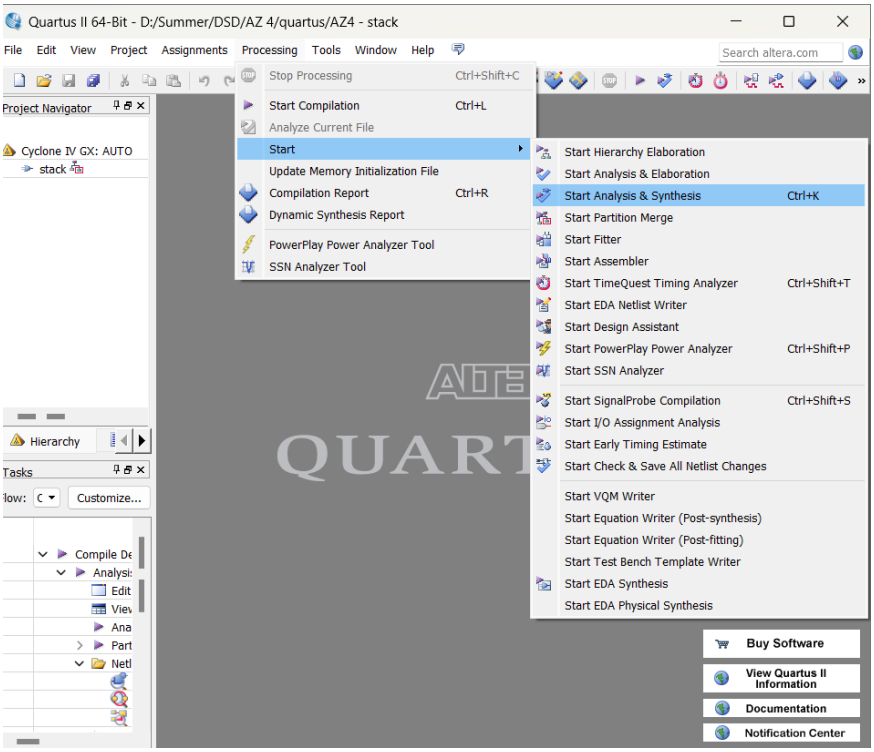


شکل ۶: شکل موج شبیه‌سازی با عمق ۴ و پهنای ۸ بیت

۷.۴ سنتز و پیاده‌سازی نرم‌افزاری

الف) همگردانی و مصرف منابع

تنها پروندهٔ سنتزپذیر طرح، یعنی stack.v، در Quartus II 13.0.1 و برای خانوادهٔ Cyclone IV GX همگردانی شد. انتخاب قطعه روی حالت خودکار گذاشته شد و ابزار قطعهٔ EP4CGX15BF14C6 را برگزید. همگردانی از مسیر Processing > Start آغاز شد.



شکل ۷: آغاز فرایند همگردانی و سنتز در Quartus II

خلاصهٔ گزارش پس از مرحلهٔ تحلیل و سنتز، برآورد اوّلیّهٔ ۷۷ عنصر منطقی را نشان می‌دهد. در این مرحله هنوز جانمایی انجام نشده و قطعهٔ هدف نیز نهایی نشده است؛ ازاین‌رو درصد مصرف منابع گزارش نمی‌شود.

Flow Summary	
Flow Status	Successful - Wed Aug 12 17:47:40 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	stack
Top-level Entity Name	stack
Family	Cyclone IV GX
Total logic elements	77
Total combinational functions	45
Dedicated logic registers	40
Total registers	40
Total pins	14
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0
Total GXB Receiver Channel PMA	0
Total GXB Transmitter Channel PCS	0
Total GXB Transmitter Channel PMA	0
Total PLLs	0

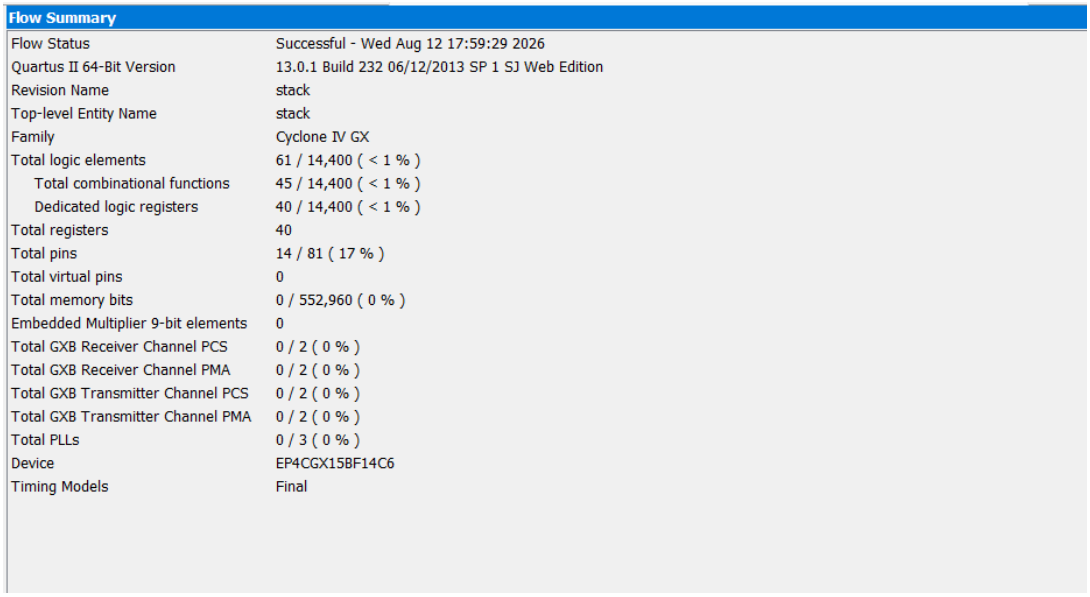
شکل ۸: خلاصه‌ی گزارش پس از تحلیل و سنتز

پس از همگردانی کامل، جانمایی و مسیریابی نیز انجام می‌شود و ابزار می‌تواند بخشی از توابع ترکیبی و ثبات‌ها را در یک عنصر منطقی مشترک بسته‌بندی کند. به همین دلیل شمار عناصر منطقی در گزارش نهایی کمتر از برآورد مرحلهٔ سنتز است.

Compilation Report - stack

Flow Summary	
Flow Status	Successful - Wed Aug 12 17:49:15 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	stack
Top-level Entity Name	stack
Family	Cyclone IV GX
Total logic elements	68 / 14,400 (< 1 %)
Total combinational functions	45 / 14,400 (< 1 %)
Dedicated logic registers	40 / 14,400 (< 1 %)
Total registers	40
Total pins	14 / 81 (17 %)
Total virtual pins	0
Total memory bits	0 / 552,960 (0 %)
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0 / 2 (0 %)
Total GXB Receiver Channel PMA	0 / 2 (0 %)
Total GXB Transmitter Channel PCS	0 / 2 (0 %)
Total GXB Transmitter Channel PMA	0 / 2 (0 %)
Total PLLs	0 / 3 (0 %)
Device	EP4CGX15BF14C6
Timing Models	Final

شکل ۹: خلاصه‌ی گزارش پس از نخستین همگردانی کامل



Flow Summary	
Flow Status	Successful - Wed Aug 12 17:59:29 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	stack
Top-level Entity Name	stack
Family	Cyclone IV GX
Total logic elements	61 / 14,400 (< 1 %)
Total combinational functions	45 / 14,400 (< 1 %)
Dedicated logic registers	40 / 14,400 (< 1 %)
Total registers	40
Total pins	14 / 81 (17 %)
Total virtual pins	0
Total memory bits	0 / 552,960 (0 %)
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0 / 2 (0 %)
Total GXB Receiver Channel PMA	0 / 2 (0 %)
Total GXB Transmitter Channel PCS	0 / 2 (0 %)
Total GXB Transmitter Channel PMA	0 / 2 (0 %)
Total PLLs	0 / 3 (0 %)
Device	EP4CGX15BF14C6
Timing Models	Final

شکل ۱۰: خلاصه‌ی گزارش در آخرین همگردانی کامل؛ مرجع اعداد این گزارش

مرجع اعداد این گزارش، آخرین همگردانی کامل است: ۶۱ عنصر منطقی شامل ۴۵ تابع ترکیبی و ۴۰ ثبات اختصاصی، و ۱۴ پایه. مصرف عناصر منطقی کمتر از یک درصد ظرفیّت قطعه و مصرف پایه‌ها ۱۷ درصد است. اختلاف اندک شمار عناصر منطقی میان دو اجرای کامل، از تفاوت بسته‌بندی و جانمایی در هر اجرا ناشی می‌شود و ماهیّت طرح را تغییر نمی‌دهد.

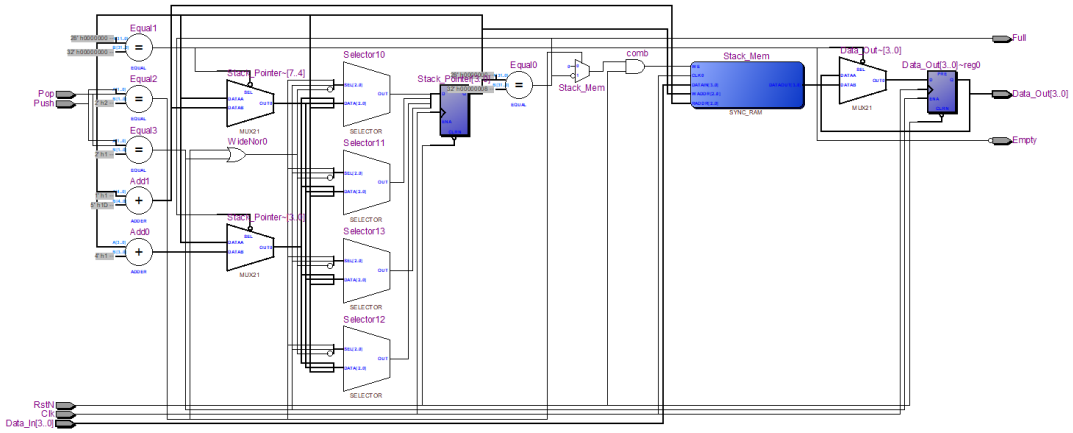
دو عدد این گزارش را می‌توان مستقیماً از روی کد راستی‌آزمایی کرد. شمار ثبات‌ها باید برابر باشد با هشت خانهٔ چهاربیتی حافظه، چهار بیت اشاره‌گر و چهار بیت Data_Out:

$$8 \times 4 + 4 + 4 = 40,$$

که دقیقاً با عدد گزارش‌شده می‌خواند. شمار پایه‌ها نیز برابر است با هشت پایهٔ ورودی، یعنی ساعت، بازنشانی، چهار بیت دادهٔ ورودی و دو فرمان، به‌علاوهٔ شش پایهٔ خروجی، یعنی چهار بیت دادهٔ خروجی و دو نشانک؛ در مجموع ۱۴ پایه.

ب) نماهای سطح انتقال ثبات‌ها و فناوری

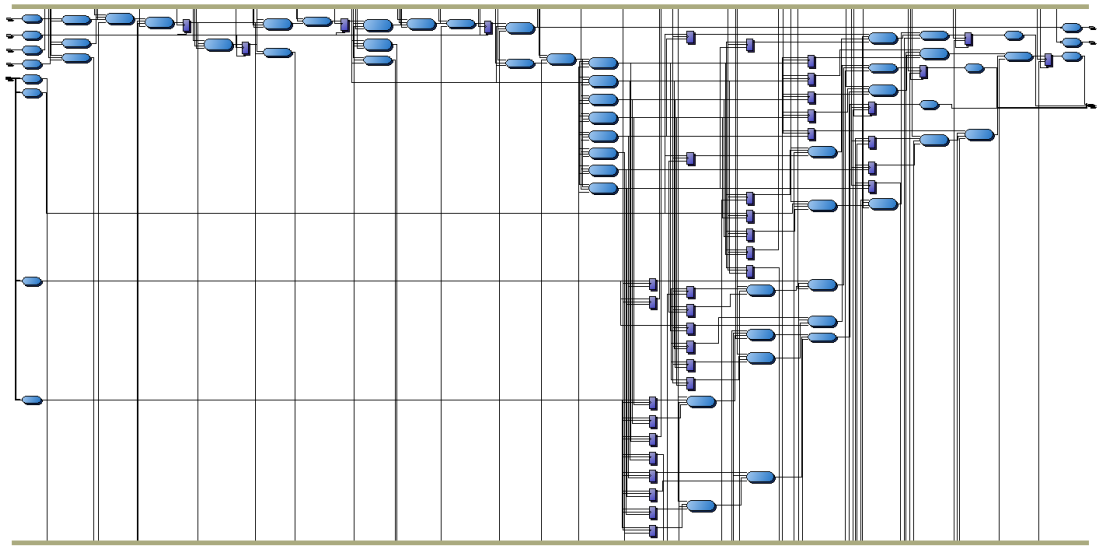
نمای سطح انتقال ثبات‌ها ساختار مورد انتظار را نشان می‌دهد: آرایهٔ حافظه به‌صورت یک بلوک با برچسب SYNC_RAM، ثبات اشاره‌گر به‌همراه دو جمع‌کننده برای افزایش و کاهش آن، مقایسه‌کننده‌های تولید نشانک‌ها و ثبات خروجی Data_Out. مسیر فرمان‌ها از پایه‌های Push و Pop به تسهیم‌کننده‌های انتخاب مقدار بعدی اشاره‌گر نیز به‌روشنی دیده می‌شود.



شکل ۱۱: نمای سطح انتقال ثبات‌های پشته

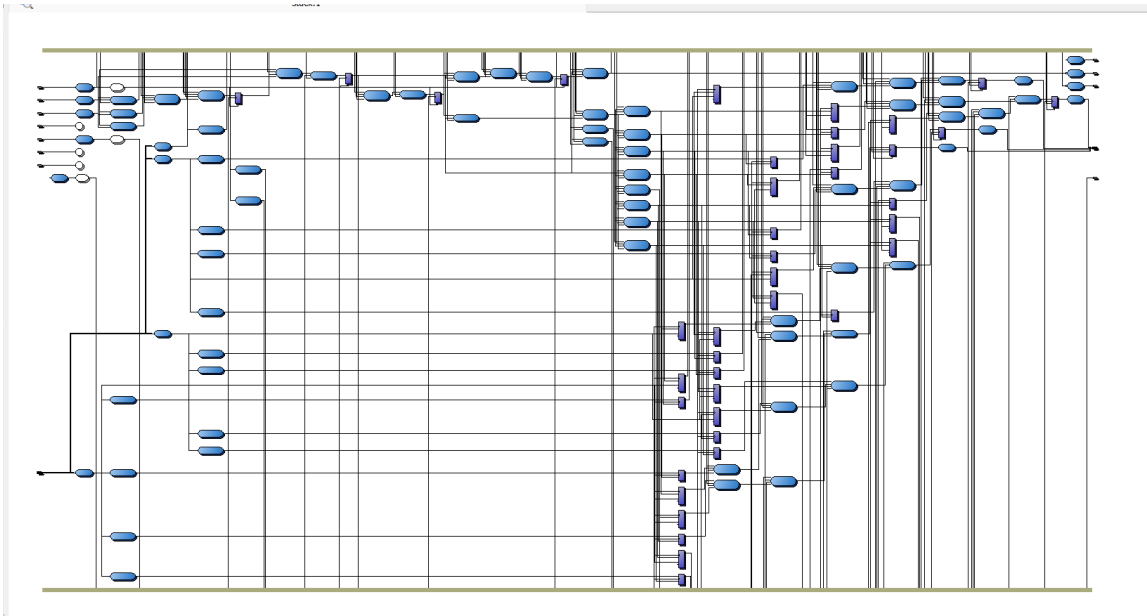
نکتهٔ درخور توجّه آن است که هرچند آرایهٔ حافظه در این نما به‌صورت حافظهٔ هم‌گام[۱۵] استنتاج شده است، گزارش منابع مقدار بیت‌های حافظهٔ مصرف‌شده را صفر اعلام می‌کند. دلیل آن است که کلّ حافظهٔ پشته تنها $8 \times 4 = 32$ بیت است و ابزار ترجیح داده به‌جای اشغال یک بلوک حافظهٔ اختصاصی، آن را با ثبات‌های معمولی پیاده کند؛ همان ۴۰ ثباتی که در گزارش منابع دیده می‌شود.

نماهای فناوری پس از نگاشت و پس از جانمایی نیز نشان می‌دهند که این ساختار به جدول‌های جست‌وجو و فلیپ‌فلاپ‌های واقعی قطعه نگاشت و مسیریابی شده است.



شکل ۱۲: نمای Technology Map Viewer (Post-Mapping) پشته

[15] Synchronous RAM



شکل ۱۳: نمای Technology Map Viewer (Post-Fitting) پشته

ج) تحلیل زمانی

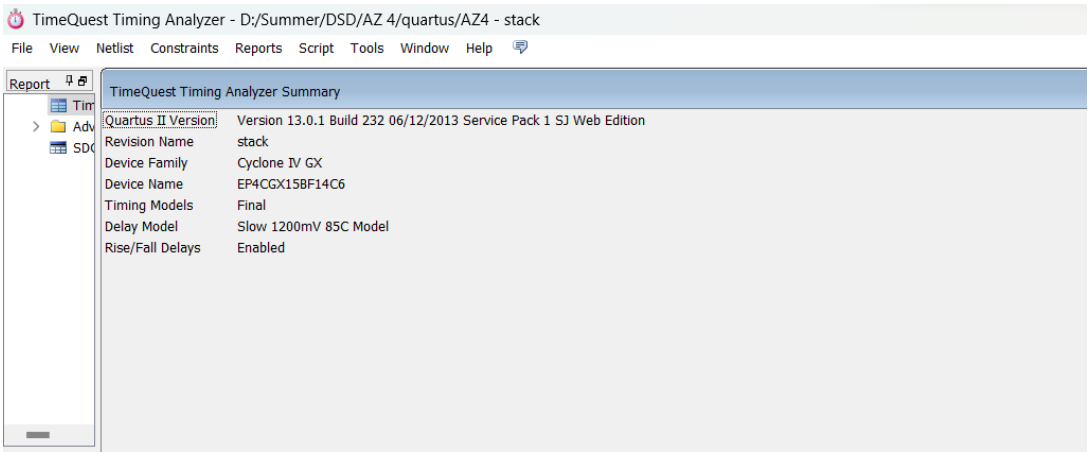
برخلاف برخی آزمایش‌های پیشین، این بار برای طرح یک پروندهٔ قیود طراحی[۱۶] نوشته شد تا تحلیل زمانی معنادار باشد. این پرونده یک ساعت با دورهٔ ۲۰ نانوثانیه، معادل ۵۰ مگاهرتز، روی پایهٔ Clk تعریف می‌کند.

```
create_clock -name Clk -period 20.000 [get_ports {Clk}]
```

کد ۶: پرونده‌ی قیود طراحی و تعریف ساعت

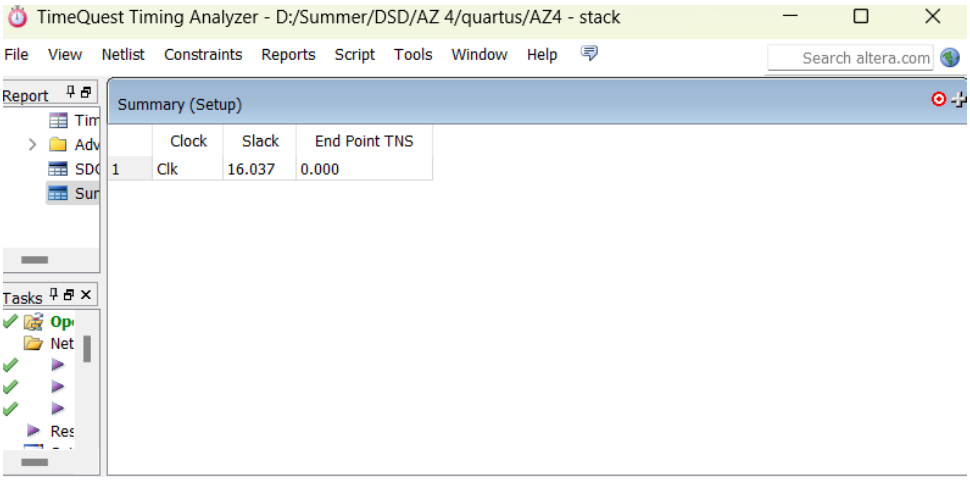
تحلیل‌گر زمانی[۱۷] با مدل کند در دمای ۸۵ درجه و ولتاژ ۱۲۰۰ میلی‌ولت، یعنی بدترین شرایط کاری، اجرا شد.

[16] Synopsys Design Constraints
[17] TimeQuest Timing Analyzer



شکل ۱۴: خلاصه‌ی تحلیل‌گر زمانی و مدل تأخیر به‌کاررفته

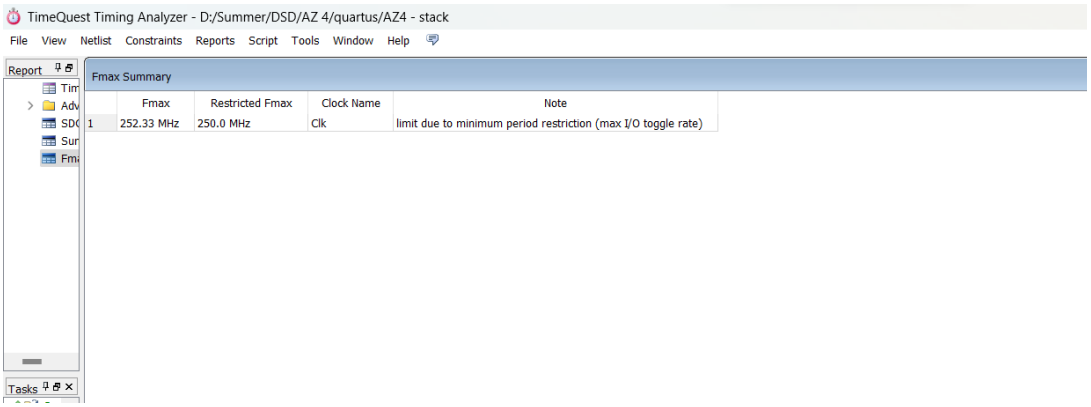
خلاصهٔ برپایی نشان می‌دهد که حاشیهٔ زمانی[۱۸] برای ساعت Clk برابر 16.037 نانوثانیه و مجموع حاشیه‌های منفی[۱۹] برابر صفر است. مثبت بودن حاشیه و صفر بودن مجموع حاشیه‌های منفی یعنی طرح قید ۵۰ مگاهرتزی را با اطمینان برآورده می‌کند و برخلاف حالتی که هیچ قیدی تعریف نشده باشد، هیچ هشدار زمانی بی‌مبنایی تولید نمی‌شود.



شکل ۱۵: خلاصه‌ی تحلیل برپایی و حاشیه‌ی زمانی مثبت

از همین اعداد می‌توان بیشینه‌بسامد[۲۰] را نیز بازسازی کرد: بلندترین مسیر داده برابر $20 - 16.037 = 3.963$ نانوثانیه است و وارون آن حدود ۲۵۲ مگاهرتز می‌شود. گزارش رسمی نیز مقدار 252.33 MHz را نشان می‌دهد.

[18] Slack
[19] Total Negative Slack
[20] Maximum Frequency (Fmax)



شکل ۱۶: گزارش بیشینه‌بسامد و بیشینه‌بسامد مقیّد

با این حال، بیشینه‌بسامد مقیّد[21] برابر 250 MHz گزارش شده است. یادداشت خودِ گزارش علّت را روشن می‌کند: این حدّ نه از منطق درونی طرح، بلکه از محدودیّت نرخ تغییر پایه‌های ورودی و خروجی[22] قطعه ناشی می‌شود. بنابراین بسامد کاری قابل‌اتّکا برای این پیاده‌سازی حدّاکثر ۲۵۰ مگاهرتز است و هم قید ۵۰ مگاهرتزی پروندهٔ قیود و هم ساعت ۱۰۰ مگاهرتزی میز آزمون، حاشیهٔ فراوان دارند.

با توجّه به برگزاری مجازی آزمایشگاه، ارزیابی این آزمایش بر پایهٔ شبیه‌سازی، سنتز و تحلیل زمانی نرم‌افزاری انجام شده و در پرونده‌های پیکربندی پروژه تخصیص پایه‌ای برای برد ثبت نشده است؛ ازاین‌رو ادّعایی دربارهٔ آزمون فیزیکی روی برد مطرح نمی‌شود.

۸.۴ نتایج

- پشته‌ای با عمق ۸ و پهنای ۴ بیت و با همان پایه‌های خواسته‌شدهٔ دستورکار، تنها با توصیف رفتاری و در یک بلوک always پیاده‌سازی شد.
- قرارداد فعّال‌صفر بودن نشانک Empty، همان‌گونه که دستورکار تصریح کرده بود، در طرح و در میز آزمون رعایت شد.
- هر چهار ترکیب فرمان‌های درج و برداشت، و هر دو حالت مرزی درج در پشتهٔ پر و برداشت از پشتهٔ خالی، آزموده شدند و مدار در همهٔ آن‌ها وضعیّت خود را سالم نگه داشت.
- ترتیب آخرین‌ورودی نخستین‌خروجی برای هر هشت مقدار درج‌شده، یک‌به‌یک و به‌صورت خودکار وارسی شد.
- آزمون فرمان هم‌زمان، افزون بر خروجی‌ها، مقدار اشاره‌گر درونی را نیز وارسی کرد تا نتیجه به‌اشتباه قبول اعلام نشود.

[21] Restricted Fmax
[22] Maximum I/O Toggle Rate

- همان کد و همان میز آزمون با عمق ۴ و پهنای ۸ بیت نیز بدون هیچ تغییری و بدون خطا اجرا شد و پارامتری بودن طرح را تأیید کرد.
- سنتز با ۶۱ عنصر منطقی، ۴۰ ثبات و ۱۴ پایه انجام شد؛ شمار ثبات‌ها و پایه‌ها دقیقاً با محاسبهٔ نظری از روی کد می‌خواند.
- تحلیل زمانی با قید ۵۰ مگاهرتزی حاشیهٔ مثبت 16.037 نانوثانیه داد و بیشینه‌بسامد 252.33 MHz با حدّ مقیّد 250 MHz گزارش شد.

نتیجه‌گیری

در این آزمایش یک پشتهٔ سخت‌افزاری با توصیف رفتاری ساخته شد. مهم‌ترین درس آن، فاصلهٔ میان «بیان رفتار» و «بیان ساختار» بود: کلّ منطق پشته در چند ده سطر و بدون هیچ اشاره‌ای به دروازه یا فلیپ‌فلاپ نوشته شد، و سنتزکننده خود آرایهٔ حافظه، شمارنده و مقایسه‌کننده‌های لازم را استنتاج کرد. در کنار آن، دقّت در جزئیّاتی که دستورکار تعیین نکرده بود — قطبیّت نشانک خالی، رفتار در برابر سرریز و تهی‌روی، و معنای فرمان هم‌زمان — تعیین‌کنندهٔ درستی طرح بود؛ چنان‌که پهنای اشاره‌گر نیز اگر یک بیت کمتر گرفته می‌شد، شرط پر بودن پشته اصلاً نمایش‌پذیر نمی‌بود. میز آزمون خودآزما با پوشش هفت سناریو، اجرای دوبارهٔ همان آزمون با پارامترهای متفاوت، و گزارش‌های سنتز و تحلیل زمانی با حاشیهٔ مثبت، در مجموع درستی عملکرد و پیاده‌سازی‌پذیری طرح را تأیید می‌کنند.